## Milestone 6 Analysis: Observability and Automated Deployment (SocialOps, Group 5)

**Monitoring.** The API writes one structured JSON log line per request (structlog) with `request_id`, `method`, `path`, `status_code` and `duration_ms`; ECS ships it to CloudWatch Logs group `/ecs/socialops-dev/api`. A metric filter with pattern `{ $.status_code >= 500 }` adds 1 to custom metric `Api5xxCount` (namespace `SocialOps/dev`) for each matching line. An unhandled exception is logged as `request.failed` with `status_code=500`, so crashes count too, each request once. Health checks never match. The CloudWatch dashboard `socialops-dev` shows API 5xx with the alarm annotated, ALB request, target-5xx and ELB-5xx counts, API response time, ECS CPU and memory for api, web and worker, and RDS CPU.

**Alert conditions.** Alarm `socialops-dev-api-5xx` fires when `Api5xxCount` has Sum >= 1 in one 60 s period. Missing data is treated as not breaching, because a quiet minute publishes no data point. Both the ALARM and the return-to-OK transition publish to SNS topic `socialops-dev-alerts`, which emails the team. The email subscription must be confirmed once by a person. Measured on 2026-11-09 (UTC): a single 5xx counted in the 00:09 minute (Sum 1.0) took the alarm OK to ALARM at 00:10:04 and sent the email; it returned to OK at 00:16:04 when the minute had no errors, and the OK email followed. Exact request-to-email delay: [TODO, needs the request time].

**Terraform coverage.** Terraform (workspace `dev`, remote S3 state with locking) manages the VPC and network, security groups, ALB with HTTPS and the ACM certificate, ECS cluster and the api, web and worker services, RDS, Redis, S3, SQS, IAM roles, ECR, log groups, the metric filter, alarm, SNS topic and dashboard. Exceptions, with reasons: the state bucket and lock table (bootstrap stack, needed before Terraform can run); the Route 53 hosted zone (registrar nameservers depend on it); `SocialOps-Budget` (cost guard kept outside the stack it protects); Bedrock model access (granted in the console); the SNS email confirmation click; leftover Milestone 5 resources in us-east-2; and the bootstrap stack, which keeps its own state. Services use `ignore_changes = [task_definition]` so pipeline deploys are not drift. Live `terraform plan` with refresh on, run after three pipeline deploys: No changes.

**Deploy and rollback.** Every push to `main` runs lint and tests, builds and pushes images tagged with the commit SHA, then the deploy job assumes a main-only OIDC role (no stored keys). It records the running task definition revisions, registers new revisions that change only the image, runs the migration as a one-off ECS task and fails before touching any service if it exits non-zero, updates the services, waits for stability, then checks `https://api.socialops.website/health/ready` (must return `"status":"ok"`) and the web URL (200) with retries and confirms every running task is on the new revision. If any check fails, the job restores the recorded revisions, re-runs the health check to prove the old version serves, writes "ROLLED BACK" to the summary and fails the run. The ECS circuit breaker only rolls back tasks that never start; this step also catches a deployment that starts but is unhealthy. Runs: good, run 42529005106 at commit 9f643ee (api, worker and web verified on the new revisions); failed and rolled back, run 42535888765 at commit 1e89ac9 (health check failed 10/10, previous revisions restored and re-verified, run red).